МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра мережевих та інтернет технологій

СИСТЕМНЕ ПРОГРАМУВАННЯ

Лабораторна робота №2

ПАРАЛЕЛІЗМ ДАНИХ ТА ПАРАЛЕЛІЗМ ЗАДАЧ

ЧАСТИНА І

Довгодька Дениса Станіславовича

Мета: ознайомитись із основами паралелізму даних та задач на прикладі платформи C# .NET

Хід роботи

Завдання 1.

Реалізуйте програму на C#, яка створює дві задачі (Task) для паралельного виконання: перша задача виводить числа від 1 до 10 з паузою 200 мс; друга задача виводить літери від A до J з паузою 200 мс. Використайте методи Start() та WaitAll(). Яка різниця між класами Thread та Task?

Створимо програмний застосунок, який реалізує задачу. Вивід в консоль відображено на рисунку 2.1.

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Id & CurrentId)
3 - Run Task 3 (Sum & ContinueWith)
4 - Run Task 4 (Parallel.Invoke demo)
0 - Exit
Select task: 1
Task 1: Numbers & Letters
1
A
B
2
3
C
D
4
E
5
6
F
7
G
8
H
I
9
10
J
Task 1 finished.
```

Рисунок 2.1 – Виведення чисел та літер різними задачами.

Як бачимо, лог виведення між різними задачами неоднорідний, що вказує на їх паралельність виконання.

Thread представляє низькорівневий потік, який створюється і контролюється вручну, включно з його запуском, зупинкою, пріоритетом та фоновим режимом. Task є більш високорівневою абстракцією для паралельних та асинхронних обчислень, де

управління потоками здійснюється планувальником автоматично і легко інтегрується з механізмами async і await.

Завдання 2.

Створіть програму, яка запускає три задачі, кожна з яких виконує підрахунок до 5. Виведіть у консоль ідентифікатор (Id) кожної задачі та поточний ідентифікатор (CurrentId) під час виконання. Чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Маємо наступний вивід із алгоритму (рис. 2.2)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Id & CurrentId)
3 - Run Task 3 (Sum & ContinueWith)
4 - Run Task 4 (Parallel.Invoke demo)
0 - Exit
Select task: 2
Task 2: Id & CurrentId demonstration
Id of t1 = 23
Id of t2 = 24
Id of t3 = 25
Task 23 started.
Task 23 counter = 1
Task 24 started.
Task 24 counter = 1
Task 25 started.
Task 25 counter = 1
Task 25 counter = 2
Task 23 counter = 2
Task 24 counter = 2
Task 25 counter = 3
Task 23 counter = 3
Task 24 counter = 3
Task 25 counter = 4
Task 24 counter = 4
Task 23 counter = 4
Task 24 counter = 5
Task 25 counter = 5
Task 23 counter = 5
Task 25 finished.
Task 24 finished.
Task 23 finished.
Task 2 finished.

Press Enter to return to menu...
```

Рисунок 2.2 – Вивід завдання з трьома задачами та виведенням ідентифікаторами, в тому числі поточними

Ідентифікатори задач (Task Id) не гарантують впорядкованість за часом створення чи виконання. Вони унікальні для кожної задачі і можуть бути використані для відстеження виконання, логування або збереження стану конкретної задачі під час паралельного чи асинхронного виконання.

Завдання 3.

Реалізуйте програму, яка:

- створює задачу для обчислення суми чисел від 1 до N (N вводиться користувачем).
- використовує метод ContinueWith() для запуску другої задачі, яка виводить результат у консоль.
- використовує лямбда-вираз для реалізації продовження.

Які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Отримаємо наступний вивід результату виконання створеного алгоритму для вирішення задачі (рис. 2.3)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Id & CurrentId)
3 - Run Task 3 (Sum & ContinueWith)
4 - Run Task 4 (Parallel.Invoke demo)
0 - Exit
Select task: 3
Enter N: 5
Sum from 1 to 5 = 15
Task 3 finished.

Press Enter to return to menu...
```

Рисунок 2.3 – Результат роботи алгоритму

Звернемо увагу на код (рис. 2.4), який відповідає встановленим вимогам завдання

```
static void RunTask3()
{
    Console.Write("Enter N: ");
    int N = int.Parse(Console.ReadLine());

    Task<int> t1 = Task.Factory.StartNew(SumTask, N);

    Task t2 = t1.ContinueWith(task =>
    {
        Console.WriteLine($"Sum from 1 to {N} = {task.Result}");
    });

    t2.Wait();
    Console.WriteLine("Task 3 finished.");
}
```

Рисунок 2.4 – Код методу розрахунку суми чисел

Використання продовжень задач дозволяє виконувати код після завершення задачі без блокування головного потоку, спрощує управління асинхронними операціями та зменшує ризик дедлоків порівняно з класичними методами синхронізації, такими як Join.

Завдання 4.

Розробіть програму, яка виконує три методи паралельно за допомогою Parallel.Invoke():

- метод 1: обчислює факторіал числа.
- метод 2: обчислює суму чисел від 1 до N.
- метод 3: виводить повідомлення у консоль з паузою 300 мс.

Які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?

Виведемо результат роботи програми (рис. 2.5)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Id & CurrentId)
3 - Run Task 3 (Sum & ContinueWith)
4 - Run Task 4 (Parallel.Invoke demo)
0 - Exit
Select task: 4
Task 4: Parallel.Invoke demonstration
[Thread 1]: Factorial of 5 = 120
[Thread 5]: Message from Task 4
[Thread 6]: Sum from 1 to 5 = 15
[Thread 5]: Message from Task 4
[Thread 5]: Message from Task 4
[Thread 5]: Message from Task 4
[Thread 5]: Message from Task 4
Task 4 finished.
```

Рисунок 2.5 – Виконання алгоритмічних задач різними Task програми із використанням Parallel.Invoke()

Parallel.Invoke() дозволяє запускати кілька методів паралельно без явного створення задач або потоків, автоматично керує їх виконанням через пул потоків, забезпечує баланс навантаження та економить ресурси, тоді як при створенні окремих задач доводиться самостійно планувати виконання та синхронізацію.

Висновок: В ході лабораторної роботи було досліджено роботу потоків та задач у C#. Показано, що Task є високорівневою абстракцією для асинхронних та паралельних обчислень з автоматичним плануванням, тоді як Parallel.Invoke дозволяє запускати кілька дій одночасно без явного створення задач, ефективно використовуючи пул потоків і спрощуючи синхронізацію. Обидва підходи дозволяють організувати паралельне виконання коду і підвищують продуктивність програми.